

Neural Processing Unit (NPU)

1. Overview

This document describes the architecture of a Neural Processing Unit (NPU) designed to accelerate Convolutional Neural Network (CNN) inference. The current implementation supports a single convolutional layer with three filters and a single stride. The design uses INT8 precision for activations and weights, with INT16 precision for accumulation, and employs 16 activation loads and 9 weight loads.

Parameter	Value
Network type	CNN-based architecture
Convolution layer	1 layer, 3 filters, single stride
Activation loads	16
Weight loads	9
Activation / weight precision	INT8
Accumulation precision	INT16
Processing elements (PEs)	3

2. Finite State Machine (FSM) Architecture

The NPU control logic is organized as a hierarchical finite state machine with three levels: layer, operation, and sub-operation.

Layer State

- `rst_lay` — Reset state for the layer level
- `l1` — Layer 1 (active convolutional layer)

Operation State

- `rst_op` — Reset state for the operation level
- `conv` — Convolution operation
- `max_pool` — Max-pooling operation

Sub-operation State

- `rst_subop` — Reset state for the sub-operation level
- `al` — Activation load
- `wl` — Weight load
- `mac` — Multiply-accumulate operation

While the reset signal is asserted, the layer, operation, and sub-operation states all remain in their respective reset states. Once reset is de-asserted and the start signal is asserted, the FSM begins normal operation.

3. Weight and Activation Loading

The `al` (activation load) and `wl` (weight load) sub-operations are responsible for transferring activation and weight values into the NPU's local memory ahead of computation.

4. Multiply-Accumulate (MAC) Operation

The design uses three processing elements (PEs) operating in parallel. During the `mac` sub-operation, each PE is loaded with the weights of its respective filter, and activations are loaded over the first three clock cycles.

After the initial load, weight loading is held constant while activations continue to flow through the array with a stride of one, using a sliding window of three activation values. Multiplication is performed between the buffered activations and the stationary weights, which allows data to be reused across the convolution window rather than reloaded for every step — improving both throughput and memory efficiency.

Each PE additionally implements a zero-skipping optimization: if either operand of the multiplication (weight or activation) is zero, the multiplication is skipped and the PE returns zero directly. This reduces switching activity and computation power for sparse inputs or weights.

5. Max-Pooling and ReLU

The max_pool operation combines ReLU activation and max-pooling in a single stage. Two successive MAC output values are compared, and the larger (post-ReLU) value is written back to the same local memory location that held the MAC result. Reusing this memory location for the pooled output avoids allocating separate storage for pooling results, reducing overall memory usage.

Once the max-pooling stage completes, a stop signal is asserted to indicate that the operation has finished.

6. Summary

The NPU pipeline proceeds from reset, through activation and weight loading, into the multiply-accumulate stage with data reuse and zero-skipping optimizations, and finally into a combined ReLU and max-pooling stage that reuses MAC memory before signaling completion via the stop signal.

7. Note

This design is a functional and timing model built to study FSM control flow and cycle-accurate data movement in a CNN accelerator; it has not been trained, tested, or validated against real AI/ECG data or benchmarked for classification accuracy.